Software Requirements Specification

Assignment 1 — Full Team Deliverable

Team: CU#

UBB-SE-2026

2026

Module	Owner
Main Catalog & Filtering	**Petru**
External Reviews Dashboard	**Filip**
Standard Movie Rating & Review	**Bogdan**
Points & Badge System	**Lucas**
Weekly Battle	**Stefana**
Movie Forum	David
Voting and Discovery	Tudor

1. Main Catalog & Personalized Filtering

Owner: CU# Team

1.1 Core Catalog Display

ID	REQ-1.1	**Owner**	**Petru**
Requirement	The system shall display a primary catalog of available movies.		
Details	Each entry in the catalog must present essential movie metadata, including the title, release year, movie poster, and a summarized internal rating.		
Verifiability	A user navigating to the main view can see a populated list or grid of movie entities with all required metadata fields visible.		

1.2 Dynamic Filtering Engine

ID	REQ-1.2	**Owner**	**Petru**
Requirement	The system shall allow users to filter the displayed movie catalog based on specific metadata attributes combined with searched words.		
Details	The interface must provide selectable options to filter the catalog by Genre, associated Actors, and a minimum Review Score. A text search field must also be available.		
Verifiability	Selecting a filter option updates the catalog view to exclusively show movies matching the selected criteria. Combining multiple filters narrows results further.		

1.3 Cross-Team Integration: Personalized Recommendations

ID	REQ-1.3	**Owner**	**Petru**
Requirement	The system shall order the movie catalog based on the logged-in user's individual preferences.		
Details	The system will query the Profiling Team's data to extract the user's preferred genres and actors, applying a sorting algorithm to prioritize matching movies. The user shall have a checkbox option to enable/disable personality-based sorting.		
Verifiability	Enabling the personalization checkbox reorders the catalog based on user preference data; disabling it reverts to default ordering.		
Cross-Team	Requires read access to user preference data managed by the User Personality and Profiling Team.		

1.4 Cross-Team Integration: Entity Sharing (Movies Table)

ID	REQ-1.4	Owner	**Petru**
Requirement	The system shall act as the primary authoritative source for the Movies data entities within the shared application database.		
Details	The MovieId primary key will be used as a foreign key by the Rentals Team and Movie Events Team to link rental transactions and events to the correct film entities.		
Verifiability	Other teams can successfully create records referencing MovieId foreign keys without integrity violations.		
Cross-Team	Dependency: Rentals Team and Movie Events Team must reference this team's MovieId as a foreign key.		

2. Verified External Consensus Dashboard

Owner: Filip (Review/Rating Team)

2.1 External Data Aggregation

ID	REQ-2.1	Owner	Filip
Requirement	The system will asynchronously fetch professional review data and consensus scores for a selected movie from trusted external APIs.		
Details	The data fetch must occur without blocking the main user interface. The system must normalize data from different external sources into a standard internal format.		
Verifiability	Network logs demonstrate asynchronous outbound requests to external APIs upon loading a movie details view, resulting in populated internal data structures without UI freeze.		

2.2 Consensus & Critic Review Presentation

ID	REQ-2.2	Owner	Filip
Requirement	The system shall display the aggregated external data distinct from the internal user reviews.		
Details	The system must display numerical aggregate scores (e.g., general audience score vs. critic score). It must also display a collection of individual critic reviews with the publication source, headline, a text snippet, and a navigation link to the full external article.		
Verifiability	The UI successfully renders the aggregated scores and the list of external review snippets for a given movie, visually distinct from internal user reviews.		

2.3 Lexicon Frequency Analysis

ID	REQ-2.3	Owner	Filip
Requirement	The system shall analyze the text snippets from fetched external reviews to determine the most frequently used descriptive words.		
Details	The system must filter out standard stop-words. The top recurring words will be presented visually to the user to summarize critical sentiment.		
Verifiability	The system correctly outputs the highest-frequency non-stop-words based on the provided review text. A visual representation (e.g., word cloud or ranked list) is rendered in the UI.		

2.4 Polarization Detection System

ID	REQ-2.4	Owner	Filip
Requirement	The system shall calculate the numerical difference between the professional critic aggregate score and the general audience aggregate score.		
Details	If the calculated difference exceeds a predefined threshold, the system shall display a visual indicator notifying the user that the film has highly polarized reception. If the difference is below the threshold, this indicator must remain hidden.		
Verifiability	Manually mocking a score difference above the threshold triggers the visibility of the UI indicator; mocking a score below the threshold hides it.		

3. Standard Movie Rating & Review System

Owner: Review Team

3.1 Core Rating & Review Submission

ID	REQ-3.1	**Owner**	Bogdan
Requirement	A registered user must be able to rate a movie using a 0–5 star rating system; submitting a rating doesnt require the user to also write a review.		
Details	Rating scale: 0 to 5 stars with half-star (0.5) increments. Review text: max 5000 characters. A user may rate and review a movie only once but may update or remove their submission. After a user rating, the average movie rating updates accordingly. Rating is mandatory but review is optional.		
Verifiability	A rating+review submission is stored successfully. Reviews outside the 5000 characters are rejected. The movie's average rating reflects the new submission.		

3.2 Extra Category Reviews

ID	REQ-3.2	**Owner**	Bogdan
Requirement	When a user submits a standard review, the system shall allow additional ratings(mandatory) and reviews(optional) for specific movie aspects as part of an Extra Review which are all mandatory. Extra review needs an additional minim 500 to 12000 characters review blog style.		
Details	Categories: Cinematography, Acting, CGI, Plot, Sound/Music. Each category follows the same 0–5 star (0.5 increments) rating and 50–2000 character review text rules. A user may rate each category only once but may update or remove their rating.		
Verifiability	Each category needs to be rated and reviewed. Category scores are stored and associated with the correct movie and user.		

3.3 Actor Profile Awards & Badges

ID	REQ-3.4	**Owner**	Bogdan
Requirement	The system shall generate badges for actor profiles highlighting top performers in specific genres based on user review data.		
Details	Profiles are automatically generated from aggregated category review data. Actor score is computed as the average acting rating from all reviews of movies the actor appears in for a given genre. Only actors meeting minimum movie count and rating thresholds (e.g., average >= 4.5 stars) receive badges. Profiles update automatically when new reviews are submitted.		
Verifiability	Actors meeting all threshold criteria display the correct genre badge on their profile page. Badge updates when new qualifying reviews are added.		

4. Gamification: Points & Badge System

Owner: Lucas

4.1 Objectivity Score Calculation

ID	REQ-4.1	Owner	Lucas
Requirement	The system shall calculate a weekly objectivity score for each user based on their review activity.		
Details	Scoring rules: +2 points for reviewing a film with avg rating > 3.5; -1 point for reviewing a film with avg rating < 2.0; +5 bonus for reviewing within one week of release (Early Bird).		
Verifiability	After each review submission, the user's score updates according to the formula. After 7 days of inactivity, the penalty is applied. Score never goes lower than zero.		

4.2 Badge / Achievement System

ID	REQ-4.2	Owner	Lucas
Requirement	The system shall automatically check and award achievement badges when a user meets predefined milestone conditions.		
Details	Badges: 'The Snob' (min 10 extra/mini-article reviews for films with avg rating < 5.0); 'The Super Serious' (all extra fields completed for 50 films); 'The Comedy Watcher' (over 70% of watch history is comedy); 'The Veteran' (100, 200, or 300 films watched). Badge checks run automatically after each relevant user action.		
Verifiability	When a user meets the criteria for a badge, the badge is unlocked, stored in the database, and a pop-up notification is displayed.		

4.3 Cross-Team: Points API for Betting

ID	REQ-4.3	Owner	Lucas
Requirement	The PointService shall expose methods to freeze and refund points, to be called by the Weekly Battle module.		
Details	FreezePoints(userId, amount): validates user has sufficient points, deducts them immediately, and marks them as frozen (unavailable for other use). RefundPoints(userId, amount): returns frozen points if a bet is cancelled.		
Verifiability	Calling FreezePoints with a valid amount correctly reduces the user's available balance. Calling with an amount exceeding the balance is rejected with an appropriate error.		
Cross-Team	Dependency: Weekly Battle module (Stefana) calls FreezePoints and DistributePayouts via the PointService interface.		

5. Weekly Battle System

Owner: Stefana (Betting) + Lucas (Points)

5.1 Battle Management (Admin/Backend)

ID	REQ-5.1	**Owner**	Stefana
Requirement	An automated or admin-triggered function shall select two movies each Monday to form a weekly battle.		
Details	A Battle table stores Movie_A_ID, Movie_B_ID, Start_Date, End_Date, and Status (Active/Finished). Logic must prevent unfair matchups. Only one battle may be active at a time.		
Verifiability	A new battle record is created every Monday. Only movies meeting eligibility criteria are selected. The battle status transitions from Active to Finished at end of the week.		

5.2 Betting Engine

ID	REQ-5.2	**Owner**	Stefana
Requirement	Users must be able to stake a portion of their points on one of the two battling movies.		
Details	Validation ensures the user has sufficient points. Points are deducted immediately via FreezePoints(). The user's bet is stored: User_ID, Battle_ID, Chosen_Movie_ID, Amount_Staked. A user may only place one bet per battle.		
Verifiability	Placing a valid bet deducts points and stores the record. Attempting to bet more points than available is rejected. A second bet attempt on the same battle is rejected.		
Cross-Team	Dependency: Lucas (PointService) — FreezePoints(userId, amount) must be available.		

5.3 Winner Determination & Payout

ID	REQ-5.3	**Owner**	Stefana
Requirement	At the end of the week, the system shall determine the winner based on rating increase (ΔR) during the battle period and distribute payouts.		
Details	Winner = movie with highest ΔR during battle period. Payout formula: Points Won = Stake × Multiplier (fixed or dynamic based on underdog bets). A Sunday midnight script identifies winners and triggers PointService to add points to winning users' profiles.		
Verifiability	After the battle ends, winners' point balances correctly reflect their payout. Losing bettors' frozen points are not returned.		
Cross-Team	Dependency: Lucas (PointService) — DistributePayouts must trigger point addition via PointService.		

5.4 Battle UI (Arena View)

ID	REQ-5.4	Owner	Stefana
Requirement	A dedicated page shall show the two competing movies side-by-side with a Bet Now interface.		
Details	The view displays both movies and a bet input.		
Verifiability	The Arena View renders both movies. Placing a bet from the UI correctly triggers the betting engine.		

6. Movie Forum Module

Owner: David

6.1 Thread Creation

ID	REQ-6.1	**Owner**	David
Requirement	Users must be able to create discussion threads linked to a specific movie.		
Details	The thread creation interface must allow the user to input: Title (short text field, max 500 characters), Content (long text field describing the discussion topic, max 10000) Thread contains: Title, Content, MovieId (FK) - automatically assigned based on the movie page where the thread is created, AuthorId (FK) - automatically assigned based on the logged-in user , CreationDate - automatically generated by the system, Score - initialized to 0 . Thread appears in the movie's discussion list upon creation.		
Verifiability	After submitting a valid thread, it appears in the movie's thread list. The thread is retrievable from the database with correct MovieId and AuthorId.		
Cross-Team	Dependency: MovieId from Marketplace/CU# Team; UserId from Personality Matching Team.		

6.2 Thread & Comment Viewing

ID	REQ-6.2	**Owner**	David
Requirement	Users must be able to view all threads associated with a movie and sort them using different criteria. Users must also be able to open a thread to view its full discussion.		
Details	Sorting options: Newest, Most Upvoted, Hottest (gravity algorithm). Users may click a thread to view its comments.		
Verifiability	All threads for a movie are displayed. Each sort option correctly reorders the thread list. Clicking a thread navigates to its detail view.		

6.3 Comments & Nested Replies

ID	REQ-6.3	**Owner**	David
Requirement	Users must be able to comment on threads and reply to existing comments (nested replies).		
Details	Comment contains: Content – max 10000, AuthorId - automatically assigned based on the logged-in user, ThreadId (FK) – automatically assigned based on the thread the user is commenting on, Timestamp – automatically generated by the system, Score, and optional ParentCommentId (FK, nullable) – based if the user is commenting on a comment other than a thread. Replies are displayed nested under their parent comment.		
Verifiability	A comment without a parent appears at the root level. A reply with a ParentCommentId appears nested under the correct parent comment.		

7. Voting & Discovery

Owner: Dalalau

6.4 Upvoting System

ID	REQ-6.4	**Owner**	Tudor
Requirement	Users must be able to upvote threads and comments. Each user may upvote a given item only once.		
Details	Upvoting a thread or comment increments its Score immediately. The Vote table enforces uniqueness: (UserId + ThreadId) and (UserId + CommentId) must be unique.		
Verifiability	A user can upvote an item once; a second attempt is rejected. Score updates are reflected immediately in the UI.		

6.5 Hottest Threads Algorithm

ID	REQ-6.5	**Owner**	Tudor
Requirement	The system shall calculate a HotScore for each thread to surface trending discussions.		
Details	Formula: HotScore = Score / (hoursSinceCreation + 2)^1.5. Threads are sorted by this value when the 'Hottest' option is selected.		
Verifiability	New threads with high scores appear near the top. Older threads with the same score rank lower. The sort correctly applies the gravity formula.		

8. Cross-Team Dependencies Summary

Consumer Module	Provides	Data / Method	Description
Catalog (CU#)	Personality/Profiling Team	**User preference data**	Read access to preferred genres/actors for catalog sorting.
Rentals Team	Catalog (CU#)	**MovieId (FK)**	Rental records reference MovieId from Movies table.
Events Team	Catalog (CU#)	**MovieId (FK)**	Event records reference MovieId from Movies table.
Weekly Battle (Stefana)	Points (Lucas)	**FreezePoints(userId, amount)**	Deduct & freeze points when a bet is placed.
Weekly Battle (Stefana)	Points (Lucas)	**DistributePayouts()**	Add points to winners after battle ends.
Points (Lucas)	Movies (CU#)	**MovieId, ReleaseDate**	Required for Early Bird bonus calculation.
Points (Lucas)	Users (Profiling)	**UserId**	Associate points with correct user record.
Forum Team	Movies (CU#)	**MovieId (FK)**	Forum threads reference MovieId for movie-specific discussions.
Forum Team	Users (Profiling)	**UserId (FK)**	Thread/comment AuthorId references the user system.